A

Mini Project Report

on

Student Teacher Connect App

Submitted in partial fulfillment of the requirements for the

degree

Second Year Engineering – Computer Science Engineering (Data Science)

by

Shree Bavchikar 24107009

Himanshu Joshi 24107008

Dev Gupta 24107101

Udaya Itagi 24107114

Under the guidance of

Prof. Shubhangi Soni

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING (DATA SCIENCE)

A.P. SHAH INSTITUTE OF TECHNOLOGY

G.B. Road, Kasarvadavali, Thane (W)-400615

UNIVERSITY OF MUMBAI

Academic year: 2025-26

CERTIFICATE

This to certify that the Mini Project report on **Student Teacher Connect App** has been submitted By Himanshu Joshi (24107008), Shree Bavchikar, (24107009), Dev Gupta (24107101) **And** Udaya Itagi (24107114) who are bonafide students of A. P. Shah Institute of Technology, Thane as a partial fulfillment of the requirement for the degree in **Computer Science Engineering (Data Science)**, during the academic year **2025-2026** in the satisfactory manner as per the curriculum laid down by University of Mumbai.

Prof.Shubhangi Soni
Guide

Dr. Pravin Adivarekar
HOD, CSE (Data Science)

Dr. Uttam D. Kolekar
Principal

External Examiner:
1.

Internal Examiner:
1.

Place: A. P. Shah Institute of Technology, Thane
Date:

ACKNOWLEDGEMENT

This project would not have come to fruition without the invaluable help of our guide **Prof Shubhangi Soni** Expressing gratitude towards our HoD, **Dr. Pravin Adivarekar**, and the Department of Computer Science Engineering (Data Science) for providing us with the opportunity as well as the support required to pursue this project. We would also like to thank our project coordinator **Prof. Hardiki Patil and Prof. Dipali Gatt** who gave us her valuable suggestions and ideas when we were in need of them. We would also like to thank our peers for their helpful suggestions.

TABLE OF CONTENTS

1. Introduction……..1

   1.1.Purpose.....1

   1.2.Problem Statement……...2

   1.3.Objectives..... 2

   1.4.Scope.....…..2

2. Proposed System…….....3

   2.1.System Architecture …….……...3

   2.2.Features and Functionality……...4

3. Project Outcomes.....…....5

4. Software Requirements.....7

5. Project Design……...…….....9

6. Project Scheduling.....11

7. Results…….14

8. Conclusion…….....20

References

Chapter 1

Introduction

In any academic institution, effective communication between students and faculty is essential for a productive learning environment. However, students often face difficulties in locating faculty members due to the absence of a centralized system to show real-time availability. This leads to wasted time, missed opportunities for doubt-clearing, and inefficient coordination. To address these challenges, this project presents the Student Teacher Connect, a mobile application that enables teachers to update their availability status and allows students to send structured meeting requests with instant notifications. The system aims to streamline, reduce search time, and improve overall academic communication.

1.1. Purpose

The purpose of this document is to present the design and development of Student Teacher Connect App a mobile application aimed at bridging the communication gap between students and faculty members in a college environment. The app allows teachers to upload their timetables and manually set their availability status, while students can search for faculty members, view real-time availability, and send meeting requests. The system is intended for use by students and faculty of A.P. Shah Institute of Technology and can be extended to other educational institutions.

1.2. Objectives

• To display real-time availability status of faculty members

• To enable students to find teachers by name or department

• To facilitate booking of doubt-clearing sessions

• To reduce time wasted searching for faculty across campus

• To improve student-faculty communication efficiency

1.3. Problem Statement

Students often waste considerable time searching for faculty members across campus, unsure of their availability. There is no centralized system to view teacher timetables or current status. Scheduling doubt-clearing sessions becomes inefficient due to lack of real-time information and a structured request process. This leads to poor student-faculty communication and missed academic opportunities.

1.4. Scope

The system can be implemented in any college or university to streamline student-faculty interactions.It is useful for students to quickly find available teachers and request meetings.Teachers benefit from organized management of meeting requests and the

ability to display their timetable and status.The application can be extended to schools and coaching institutes with similar requirements.It provides a foundation for future enhancements like session feedback,calendar integration, and analytics.

Chapter 2

Proposed System

The proposed system introduces a mobile-based application designed to bridge the communication gap between students and teachers in an academic institution. The system provides a centralized platform where teachers can update their availability status and manage meeting requests, while students can easily search for faculty members and request appointments. By integrating real-time updates, role-based access, and notification services, the system ensures efficient and structured interaction between users. The proposed solution eliminates manual searching and improves overall coordination within the campus.

2.1 System Architecture

The system follows a client-server architecture with Flutter as the frontend and Firebase as the backend. Data is stored in Firestore. Profile images are stored in Firebase Storage. Push notifications are handled by Firebase Cloud Messaging, and status updates are performed manually by the teacher.

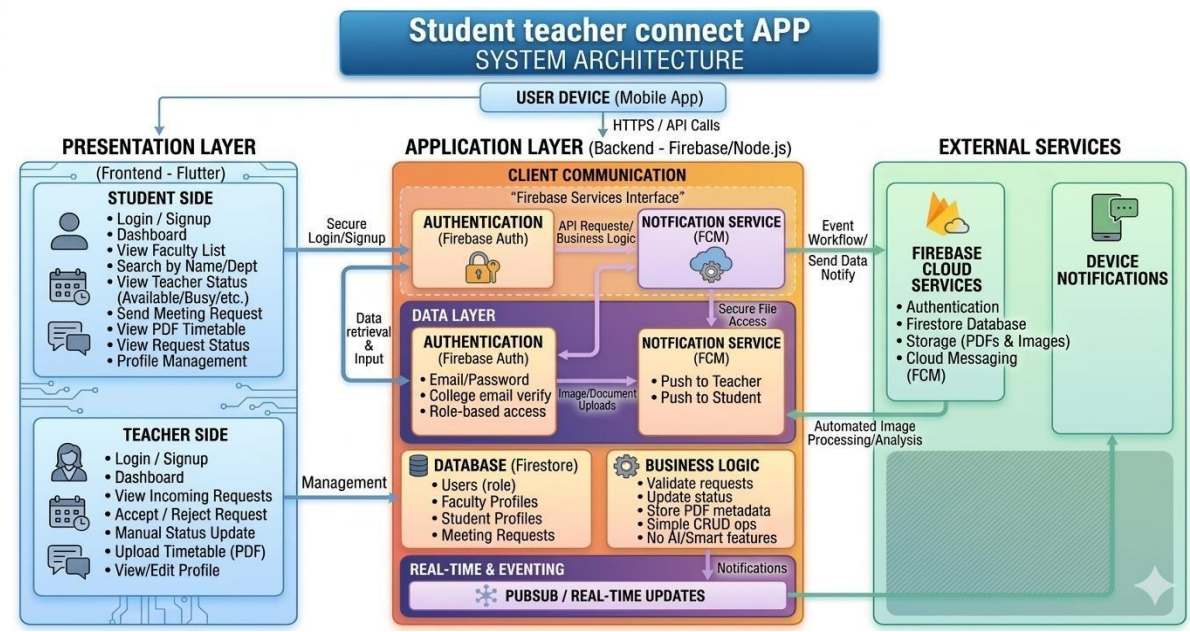

Figure 2.1: System Architecture Block and Diagram

2.2. Features and Functionality

Feature 1: Role-Based Authentication: Students and teachers log in using their college email. Firebase Authentication ensures secure access with role-based privileges.

Feature 2: Teacher Profile & Timetable Upload: Teachers can upload their weekly timetable (stored in Firebase Storage). They can also manually update their current status: Available, Busy, In Meeting, or On Leave.

Feature 3: Student Search & View: Students can search for faculty by name or department. The app displays a list of faculty with their real-time status. Students can also view the timetable of a selected teacher.

Feature 4: Meeting Request: Students can select a teacher, enter a reason, and send a meeting request. The request is stored in Firestore with status "pending".

Feature 5: Real-Time Notifications: When a student sends a request, the teacher receives a push notification via Firebase Cloud Messaging (FCM). Similarly, when a teacher accepts orrejects, the student receives a notification.

Feature 6: Request Management for Teachers: Teachers see all incoming requests in their dashboard. They can accept or reject each request, optionally adding a message. The request status is updated in the database.

Feature 7: Student Dashboard & History: Students can view the status of their requests (pending, accepted, rejected).

For accepted requests, they see any details provided by the teacher.

Feature 8: Profile Management: Both students and teachers can edit their profile information (name, email, profile picture) stored in Firestore and Firebase Storage.

Chapter 3

Project Outcomes

The successful implementation of the **Student Teacher Connect** App results in a fully functional system that enhances communication between students and faculty members. The project demonstrates the effective use of modern technologies such as Flutter and Firebase to build a real-time, user-friendly application. The outcomes highlight improved accessibility, efficient request handling, and seamless notification mechanisms, thereby solving the existing communication challenges in academic institutions.

3.1 A Fully Functional Android Mobile Application Built with Flutter

The app is developed using Flutter, which compiles natively to Android. Flutter's widget-based architecture ensures a consistent, responsive UI across different screen sizes. The final output is a standalone APK that can be installed on any Android device running API level 21 or higher.

3.2 Secure User Authentication with College Email Verification

Firebase Authentication is used to manage user sign-up and login. During registration, users must provide a valid college email, and a verification link is sent to confirm ownership. Role-based access (student/teacher) is enforced using Firebase security rules, ensuring that each user only sees features relevant to them.

3.3 Real-Time Faculty Availability Updates

Teachers can manually set their status to Available, Busy, In Meeting, or On Leave.This update is stored in Firestore and instantly reflected on the student side using Firestore's real-time listeners. Students can thus see the most current availability without refreshing the app.

3.4 A Streamlined Process for Students to Send Meeting Requests and for Teachers to Manage Them

Students select a teacher, enter a reason, and submit a request. The request is saved in Firestore with a "pending" status. Teachers view all incoming requests in a dedicated dashboard and can accept (with optional remarks) or reject with a brief reason. The status is updated, and both parties have a clear record of the request.

3.5 Push Notification System for Instant Updates on Request Status

Firebase Cloud Messaging (FCM) delivers push notifications to the respective user when a request is submitted or its status changes. Notifications work even when the app is in the background, ensuring that teachers are alerted immediately and students receive timely confirmations.

3.6 Clear Separation of Roles and Responsibilities

Students and teachers are presented with different interfaces based on their role. Students see search, request, and history features; teachers see status controls and request management. This separation is enforced both in the UI and in backend security rules, reducing complexity and improving security.

Chapter 4

Software Requirements

The development of **the Student Teacher Connect** App requires a combination of software tools and technologies that support efficient application design, development, and deployment. These tools ensure smooth functionality, real-time data handling, secure authentication, and effective user interface design. The following section outlines the software requirements used in the implementation of the system.

1) Frontend – Flutter (Dart): The frontend of the *Student Teacher Connect App* developed using Flutter, a modern UI toolkit by Google that allows building natively compiled applications from a single codebase. Flutter uses the Dart programming language and provides a rich set of customizable widgets, enabling the creation of a responsive, visually appealing, and consistent user interface across different Android devices. It also supports features like hot reload, which speeds up development and testing.

2) Backend – Firebase (Authentication, Firestore, Cloud Messaging): The backend of the application is powered by Firebase, which offers a suite of integrated cloud services. Firebase Authentication is used to provide secure user login with email verification, ensuring that only authorized users can access the system. Firestore is used for real-time data synchronization, allowing instant updates across users. Firebase Cloud Messaging (FCM) is implemented to send push notifications, ensuring that students and teachers receive immediate updates regarding meeting requests and status changes.

3) Database – Firestore: Firestore, a NoSQL cloud-based database provided by Firebase, is used to store and manage application data such as user profiles, teacher availability status, and meeting requests. It supports real-time listeners, which means that any changes made in the database are instantly reflected in the application without requiring a refresh. This feature is crucial for maintaining up-to-date information for both students and teachers.

4) Storage – Firebase Storage: Firebase Storage is used specifically for storing user profile images in the application. It provides secure and scalable cloud storage that integrates seamlessly with Firebase Authentication, ensuring that only authorized users can upload or access files. This helps in efficient management of media content while maintaining data security.

5) IDE – Visual Studio Code: Visual Studio Code is used as the primary development environment for building the application. It is a lightweight yet powerful code editor that supports multiple programming languages and provides extensions specifically for Flutter development. Features such as debugging tools, syntax highlighting, and hot reload support make the development process faster and more efficient.

6) Version Control – Git & GitHub: Git is used as the version control system to track changes in the source code, while GitHub acts as a remote repository for storing and managing the project. These tools enable collaboration among team members, maintain version history, and ensure that code changes can be easily tracked, reviewed, and reverted if necessary.

7) Documentation–Microsoft Word/GoogleDocs: Microsoft Word and Google Docs are used for preparing the project documentation. These tools support structured formatting, collaborative editing, and easy sharing. They also help ensure that the report adheres to the formatting guidelines specified by the university.

Chapter 5

Project Design

The project design phase focuses on defining the overall architecture and workflow of the system. It provides a structured representation of how different components of the application interact with each other. The design includes workflow diagrams that illustrate the flow of data and user interactions within the system. This helps in understanding the internal working and ensures efficient implementation of the proposed solution.

5.1 Work Flow

The workflow of the Student Teacher Connect App represents the sequence of operations performed by students and teachers to facilitate efficient communication and meeting scheduling. It provides a clear understanding of how users interact with the system, starting from login to final confirmation of meeting requests. The workflow ensures a structured process where students can search for faculty, send requests, and receive responses, while teachers can manage their availability and respond accordingly. The following diagram illustrates the complete flow of activities within the system

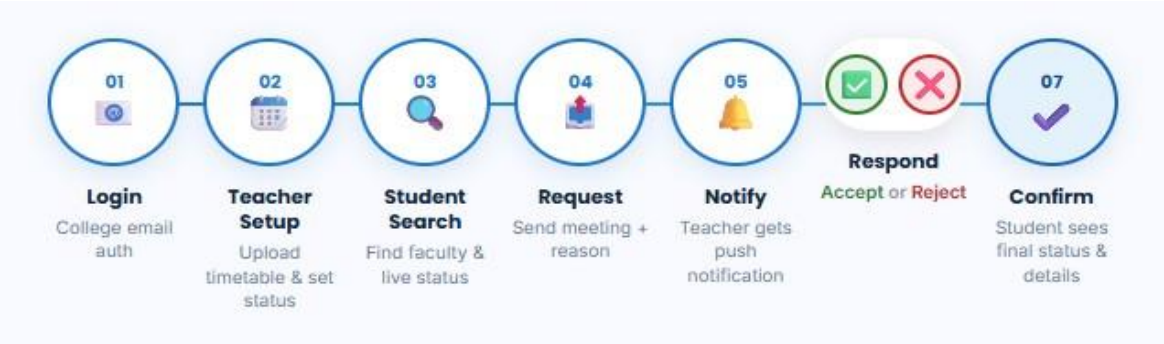

Figure 5.1: Workflow Diagram

The workflow of the **Student Teacher Connect App** illustrates the step-by-step interaction between students and teachers, ensuring smooth communication and efficient meeting management. The process begins with the Login step, where both students and teachers authenticate themselves using their college email credentials, ensuring secure and role-based access to the system.

Once logged in, teachers proceed to the Teacher Setup stage, where they upload their timetable and set their current availability status such as Available, Busy, In Meeting, or On Leave, which is stored in the database and made visible to students in real time. On the student side, users perform a Student Search, where they can find faculty members by name or department and view their live availability status along with heir timetable.

After identifying a suitable teacher, the student moves to the Request stage, where they send a meeting request by entering a reason, and the request is stored in the system with a pending status. Once submitted, the teacher receives a notification in the Notify stage through push notifications, informing them about the incoming request.

The teacher then performs an action in the Respond stage, where they can either accept or reject the request and optionally include remarks or additional details.Finally, in the Confirm stage, the student receives the response, where accepted requests display confirmed meeting details and rejected requests update the status accordingly, thus completing the workflow of the system

Chapter 6

Project Scheduling

Project scheduling is an essential aspect of software development that helps in planning and organizing tasks within a defined timeline. The *Student Teacher Connect App* project follows a structured schedule divided into multiple phases such as requirement analysis, design, development, testing, and documentation. A Gantt chart is used to visually represent the timeline of activities and ensure timely completion of the project.

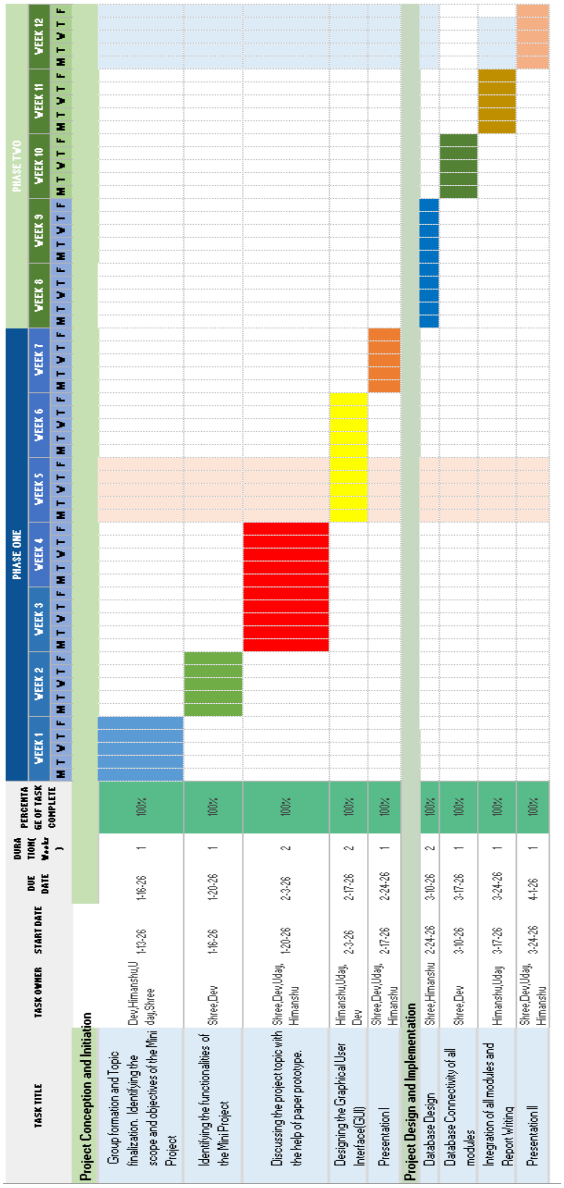

Figure 6.1: Project Gantt Chart

The Gantt chart presents a detailed timeline of the *Student Teacher Connect App* project, illustrating the sequence of activities, task allocation among team members, and the duration of each phase. The project is broadly divided into two major phases: **Project Conception and Initiation** and **Project Design and Implementation**, spanning across multiple weeks.

The project begins in **Week 1** with group formation and topic finalization, where all team members collaboratively define the project scope and objectives. This step ensures clarity of goals and proper planning from the beginning. In **Week 2**, the team focuses on identifying the functionalities of the mini project, where core features such as login, teacher status update, student search, and meeting request system are finalized.

In **Weeks 3 to 4**, the team engages in discussion of the project topic along with the help of paper prototypes. During this stage, rough sketches and conceptual models are created to visualize how the system will function. This helps in refining ideas and eliminating design issues at an early stage.

Moving forward, in **Weeks 5 to 6**, the *Graphical User Interface (GUI) design* is developed. The team designs user screens such as login page, dashboard, request forms, and status display interfaces using Flutter. This phase ensures that the application is user-friendly and visually consistent. In **Week 7**, the team conducts *Presentation I*, where the initial design and progress are reviewed.

The second phase begins with **Weeks 8 to 9**, focusing on *database design*. The structure of Firestore collections is planned, including user profiles, meeting requests, and status data. In **Weeks 10 to 11**, the development of all modules is carried out, where individual functionalities such as authentication, search, request handling, and notifications are implemented.

In **Week 11**, the team performs integration of all modules, combining frontend and backend components to ensure the system works as a complete application. Finally, in **Week 12**, report writing and final presentation are completed, documenting the entire project and demonstrating the working system.

Each task is assigned to specific team members, ensuring equal contribution and accountability. The Gantt chart also reflects 100% completion of all tasks, indicating that the project was executed successfully within the planned timeline. Overall, this structured scheduling approach helps in efficient time management, task tracking, and smooth execution of the project.

Chapter 7

RESULT

The results of the **Student Teacher Connect App** demonstrate the successful implementation of all major features of the system. The application was tested under various scenarios to verify its functionality, usability, and performance. The following section presents screenshots of the application along with explanations, showcasing key features such as login, teacher status updates, student dashboard, meeting requests, and notification system.

Key screens of the App

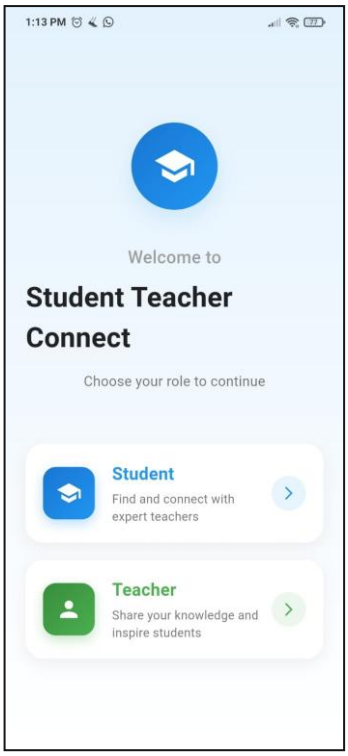

Figure 7.1: Welcome and Role Selection Screen

The figure shows the initial screen of the **Student Teacher Connect App**, where users select their role as either Student or Teacher. This role-based selection directs users to their respective functionalities within the application. Students can search for teachers and send requests, while teachers can manage availability and respond to requests. The interface is simple and user-friendly, ensuring easy navigation.

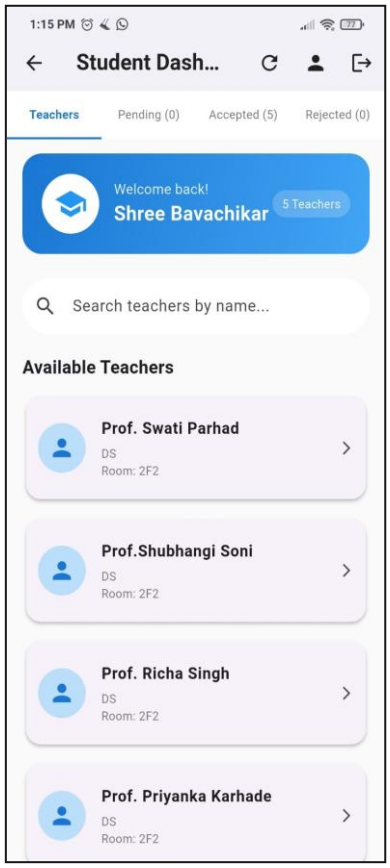

Figure 7.2: Student Dashboard – Teacher List Interface

The provided image (consider labeling it as Figure 7.2: Student Dashboard – Teacher List Interface) displays a section of a student dashboard focused on teacher information. The interface includes a welcome message for a student named "Shree Bavachikar," indicating there are 5 teachers in total, along with a search bar to filter teachers by name. Below, an “Available Teachers” list shows two entries: Prof. Swati Parhad and Prof. Shubhangi Soni. For each teacher, the dashboard shows an unspecified ID field (left blank) and a room number (2F2 for both), along with a right-arrow icon (">") likely for viewing more details. Overall, this view helps students quickly locate teacher details and navigate to individual profiles.

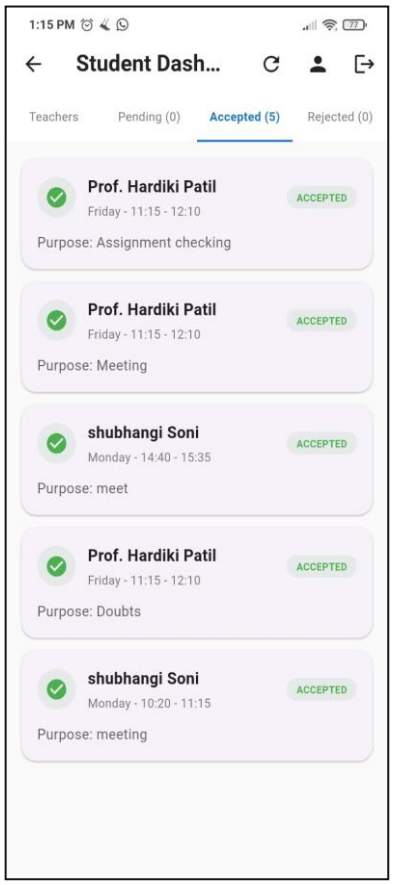

Figure 7.3: Student Dashboard – List of Accepted Appointment Slots

This screen shows multiple accepted meeting requests, all with the status [ACCEPTED]. Most entries feature **Prof. Hardiki Patil** on Friday from 11:15–12:10, with purposes including "Assignment checking," "Meeting," and "Doubts." Two entries are for **Shubhangi Soni** – one on Monday 14:40–15:35 (purpose: "meet") and another on Monday 10:20–11:15 (purpose: "meeting"). The layout allows a student to see confirmed appointments along with time, teacher name, and purpose.

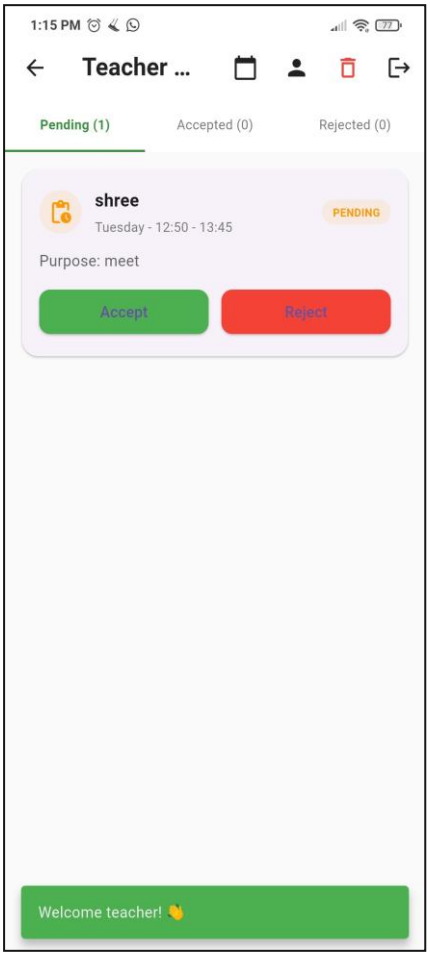

Figure 7.4: Teacher Interface – Pending Appointment Request

This mobile-style view (timestamp 1:15 PM) is from a teacher's perspective, as indicated by the greeting "Welcome teacher! ." Tabs show **Pending (1)**, Accepted (0), and Rejected (0). A single pending request from **Shree** appears for Tuesday 12:50–13:45 with purpose "meet." The teacher can **[Accept]** or **[Reject]** the request. This interface helps teachers manage incoming student meeting requests efficiently.

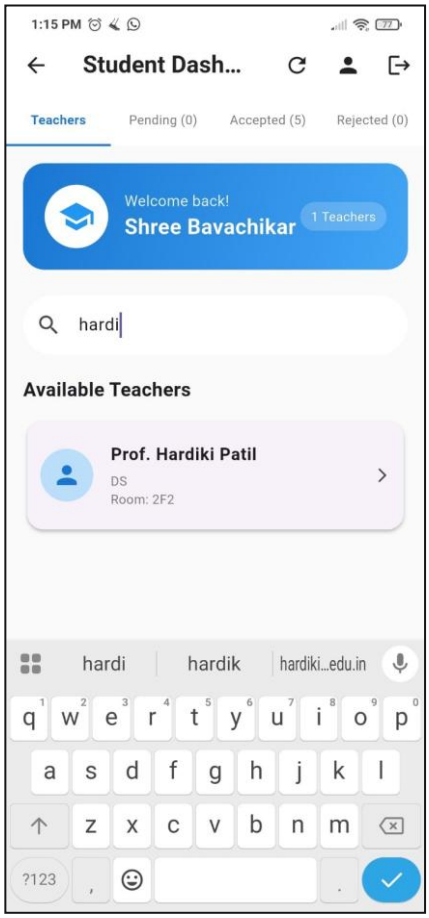

Figure 7.5: Student Dashboard – Simplified Available Teachers **View**

Returning to the student dashboard, the interface welcomes back **Shree Bavachikar** and shows only **1 Teacher** available. Under "Available Teachers," **Prof. Hardiki Patil** is listed with a blank ID field and a room number **2F2**. The right-arrow icon (not shown in text but implied) likely leads to more details or booking options. This compact view helps students quickly see which teachers are currently free for appointments.

Test Case	Expected Result	Actual Result
Student login with valid email	Access granted	Pass
Teacher updates status to "Busy"	Status changes instantly for all students	Pass
Student searches by department	Shows only relevant faculty	Pass
Student sends meeting request	Teacher receives push notification	Pass

Table 7.1: Test Cases and Results

Chapter 8

Conclusion

The **Student Teacher Connect App** effectively addresses the challenge of inefficient communication between students and faculty members within academic institutions. By providing a centralized platform for real-time availability updates and structured meeting requests, the system enhances coordination and minimizes time wastage. The application ensures transparent and reliable communication through features such as role-based access and push notifications. Unlike automated systems, it emphasizes user-controlled interactions, making it simple, practical, and easy to adopt. Overall, the project demonstrates the successful implementation of modern technologies to solve a real-world problem. In the future, the system can be further enhanced by incorporating features such as session feedback, calendar integration, and basic analytics for improved functionality.

REFERENCES

[1] Flutter Documentation. (2026). *Flutter – Build apps for any screen.* Retrieved from https://flutter.dev/docs

[2] Firebase Documentation. (2026). *Firebase – Build and run successful apps.* Retrieved from https://firebase.google.com/docs

[3] Google. (2026). *Firebase Cloud Messaging.* Retrieved from https://firebase.google.com/docs/cloud-messaging

[4] Material Design. (2026). *Material Design Guidelines.* Retrieved from https://material.io/design